

03-code-review-report

S3: Code Review Report

Revix · *driven by reviews*. What the measuring tools say about this codebase, what we did about it, and what we chose not to do.

Submission	S3 Lab work, the code quality mark
Toolkit	Radon, Xenon, Ruff, mypy, pytest with coverage
Measured on	18 September 2026, commit on <code>main</code>
Scope	43 Python source files across three packages, tests excluded
Team	Aditya Nariyapara, Devika Jonjale, Saachi Shinde
Companion	Lab work submission · Speaking script

Every figure here was produced by running the tool named beside it. Section 8 gives the exact commands, so any number in this document can be checked in under a minute rather than taken on trust.

1. The headline

Measure	Tool	Result
Maintainability index	Radon	Grade A on all 43 files , no exceptions
Cyclomatic complexity	Radon	Average A (3.90) across 344 blocks
Worst single block	Radon	D (24), and only two blocks are worse than C
Complexity ceiling	Xenon	Enforced in CI , build fails on regression
Lint and format	Ruff	Clean , enforced in CI
Static types	mypy strict	Clean , all three packages
Test coverage	pytest-cov	64% over 308 tests

The short version: nothing in this codebase grades worse than A for maintainability, 92% of all code blocks are A or B for complexity, and the two that sit at D are there for reasons we can defend rather than reasons we have not got round to.

2. Choosing the toolkit

Four tools were recommended. We used three of them, added a fourth, and left one out on purpose. A tool chosen without a reason is not a code review, it is a screenshot.

Tool	Used	Why
Radon	Yes	The only one that produces a <i>score</i> rather than a pass or fail. Cyclomatic complexity, maintainability index, raw counts and Halstead metrics, cross platform, no configuration
Ruff	Yes	Already enforced in CI before this exercise. Lint and format in one tool instead of black, isort and flake8
PyTest	Yes	308 tests, already the backbone of the project, with coverage measured in CI
Xenon	Added	Radon reports; Xenon <i>enforces</i> . See section 5, because this is the difference between knowing a number and keeping it
Mutmut	No	See below

Why we left mutation testing out

Mutmut is the most interesting tool on the list. It changes your code on purpose and asks whether any test notices, which measures something no other tool here does: not whether tests exist, but whether they would catch a bug.

We did not use it, for a reason worth stating plainly rather than hiding:

Mutmut has no native Windows support. Running it prints "*To run mutmut on Windows, please use the WSL*". All three of us develop on Windows.

A quality tool that only one environment can run is a quality tool that stops being run. We would rather report a toolkit the whole team can execute on any machine, in under a minute, than a better number that only appears when one person remembers to produce it.

What we did instead. The question mutation testing answers, "would these tests actually catch a bug", we answer with evidence rather than a score: see section 7, which lists five real defects our own tests caught before they reached anybody.

3. Maintainability index

What it measures. A single 0 to 100 score per file, derived from Halstead volume, cyclomatic complexity and lines of code. It is the closest thing the Python ecosystem has to "how hard will this file be to change in six months".

Grade	Range	Meaning	Our files
A	20 to 100	Maintainable	43
B	10 to 19	Moderate	0
C	0 to 9	Difficult	0

Every file grades A. The six lowest are worth naming, because a low A is still the place a reviewer should look first:

File	Score	Why it sits where it does
<code>pipeline/.../cli.py</code>	A (24.64)	The command line surface. Thirty-odd commands in one module, each small; the file is long rather than complicated
<code>pipeline/.../enrichment/fuse.py</code>	A (34.77)	The scoring engine. The densest logic in the project and the most heavily tested
<code>pipeline/.../evaluation/fusion_experiment.py</code>	A (39.95)	The evaluation harness, refactored during this review; see section 6
<code>pipeline/.../catalogue_discovery.py</code>	A (43.89)	Reads two sites' markup, so it carries the shape of somebody else's HTML
<code>apps/api/.../schemas.py</code>	A (45.83)	Declarative response models. Long by nature, trivial by content
<code>apps/api/.../main.py</code>	A (48.66)	Every endpoint in one module, each one an indexed read

A useful thing to notice: the three lowest scores are the three files that do the most genuine work. That is what the metric is supposed to do, and it is a reason to trust it here rather than to be alarmed by it.

4. Cyclomatic complexity

What it measures. The number of independent paths through a block of code, which is also the minimum number of test cases needed to cover it. A function scoring 24 has twenty-four ways through.

Distribution across 344 blocks

Grade	Complexity	Blocks	Share
A	1 to 5	272	79.1%
B	6 to 10	44	12.8%
C	11 to 20	26	7.6%
D	21 to 30	2	0.6%
E, F	31+	0	0%

Average: A (3.90). Nearly four fifths of the codebase is in the simplest band, and there is nothing in the two worst bands at all.

The two blocks at D

Both were examined during this review and both were deliberately left alone.

`fuse_variant`, **D (24)** in `enrichment/fuse.py`. The scoring engine: it weights every piece of evidence, computes the bootstrap confidence interval, applies the evidence floor, decides suppression, and writes the claims with their citations. Its complexity is the domain's, not an accident of style, and splitting it into five functions that are only ever called in sequence would move the complexity into the call graph rather than remove it. It is the most heavily tested function in the project.

`run_connector`, **D (21)** in `connectors/runner.py`. The ingest loop, which has to hold three failure modes open at once: a missing credential, an open circuit breaker, and an unexpected exception, each recorded differently so that one source failing never stops the others. That branching *is* the resilience requirement.

Why not refactor them anyway. Both are on the critical write path, days before a submission, and a refactor that changes a verdict is worse than a D grade. The honest engineering position is that a threshold should be set where the code is and then defended, not chased with changes that trade a number for risk.

5. Turning the score into a gate

Radon will tell you a function scores E and then happily let you merge it. That is a report, not a control.

So **Xenon runs in CI and fails the build** on any regression:

```
xenon --max-absolute D --max-modules B --max-average A
```

Threshold	Set to	Why
Any single block	D	Where the code actually is. Nothing may get worse than the two functions above
Any single module	B	A module may contain a hard function; it may not become a hard module
The whole codebase	A	The average must stay in the simplest band

This is the part of the exercise we would defend hardest. **A measurement taken once is a snapshot; a measurement wired to an exit code is a property.** The grades in this document cannot silently decay, because the next pull request that makes them worse does not merge.

6. What the review changed

The measurement found one block materially worse than everything else:

```
_evaluate  E (33)  pipeline/src/revix_pipeline/evaluation/fusion_experiment.py
```

The only E in the codebase, and 40% worse than the next worst block. Reading it, the cause was obvious: it did two unrelated jobs in one function. It drew samples from the corpus and recorded what each weighting strategy made of each draw, and then it turned all of those observations into summary statistics. Neither half is complicated. Doing both in one place was.

Split into five functions along that seam: `_draw`, `_score_one_draw`, `_rank_correlation`, `_calibration` and `_summarise`, with a small `_Trials` value carrying the observations between the two halves.

Before	After	
Worst block in the codebase	E (33)	D (24)
Blocks graded E or worse	1	0
Average complexity	A (3.944)	A (3.895)

The thirteen tests covering the fusion experiment passed unchanged throughout, which is the only reason a refactor of this kind is safe to make at all.

This is the part of the report worth reading as method rather than result: **measure, find the worst thing, understand why it is the worst thing, fix the one that is safe to fix, and document the ones that are not.**

7. What the tests are actually worth

Coverage is **64% over 308 tests**, which is a number that flatters or deceives depending on what the tests do. The honest measure of a suite is what it has caught, so here is the list:

Defect caught	Why it mattered
Macro-F1 averaged over zero-support aspects	A perfect system scored 0.22 on a two-topic set. Our evaluation was lying to us in our own favour
A connector recorded our own search term as the source's answer	We would have been citing ourselves as evidence
An unordered <code>LIMIT</code> in fusion	Ingest and fusion silently disagreed about which variant they meant
Security headers added inside the rate limiter	A 429, the response most likely to reach a hostile client, was the one going out bare
A migration emitting <code>NOT NULL</code> with no default	Would have failed against the 129 verdict rows already in production

Coverage is deliberately not gated in CI. A coverage threshold is trivially satisfied by tests that execute code without asserting anything about it, and we would rather the number stayed honest than stayed high.

8. Raw metrics, and reproducing all of this

Raw counts (Radon)

Measure	Value
Lines of code	9,991
Logical lines	4,795
Source lines	6,646
Comment lines	845
Docstring and block comment lines	1,073
Blank lines	1,435
Comment and docstring density	19% of all lines

That last figure is the one we would point at. Roughly one line in five explains something, and house style is that a comment states a decision rather than restating the code.

Commands

```
# The score: maintainability index, per file
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s

# Cyclomatic complexity, average plus anything above B
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC

# Raw counts
uv run radon raw packages/revix_core/src pipeline/src apps/api/src -s

# The gate, which is what CI runs
uv run xenon --max-absolute D --max-modules B --max-average A \
  packages/revix_core/src pipeline/src apps/api/src

# The rest of the quality gate
uv run ruff check . && uv run ruff format --check .
uv run mypy packages/revix_core/src pipeline/src apps/api/src
uv run pytest --cov --cov-report=term
```

Every one of these also runs on every pull request. The metrics are printed in the CI log under **Code metrics**, and the gate is the step named **Complexity gate**.

9. Summary for the one-to-one

If there is time for only four sentences:

1. **Every file grades A for maintainability, and the average cyclomatic complexity is A at 3.90 across 344 blocks**, with nothing above D.
2. **The review changed the code**: the single E-grade function was found, understood, split into five, and the codebase now has no E at all.
3. **The score is enforced, not just reported**. Xenon fails the build if any of it regresses, which is the difference between a measurement and a property.
4. **We left mutation testing out and said why**: mutmut cannot run natively on Windows, all three of us are on Windows, and a tool nobody can run is not a control.